Aprendizaje de Máquina
Sesión 15: Reducción de dimensiones

Angel Rodríguez

¿Por qué reducir dimensiones?

Muchos conjuntos de datos tienen miles o millones de características.

Beneficios

- Entrenamiento más rápido.
- Menor uso de memoria y disco.
- Menos ruido o redundancia.
- Visualización en 2D o 3D.

Riesgos

- Se puede perder información útil.
- Puede empeorar el desempeño.
- Agrega un paso extra al pipeline.
- No siempre ayuda a redes neuronales profundas.

La maldición de la dimensionalidad

Nuestra intuición de 2D/3D falla en espacios de muchas dimensiones.

- En dimensiones altas, los puntos aleatorios tienden a estar lejos entre sí.
- Los datos se vuelven más **dispersos**.
- Métodos basados en distancia, como k-NN, pueden volverse menos confiables.
- Para mantener la misma densidad, el número de muestras necesarias crece exponencialmente.

Idea clave

Más dimensiones no significan automáticamente más información útil; también pueden significar más espacio vacío.

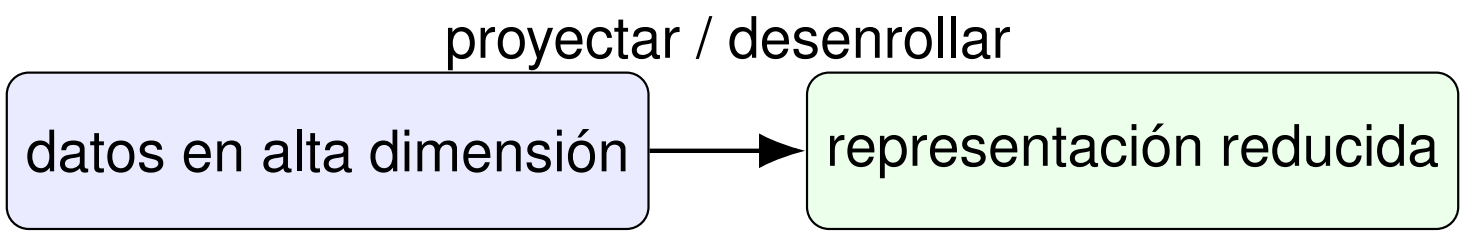
Dos enfoques principales

1. Proyección

- Los datos están cerca de un subespacio de menor dimensión.
- Se proyectan sobre ese subespacio.
- Ejemplo: PCA.

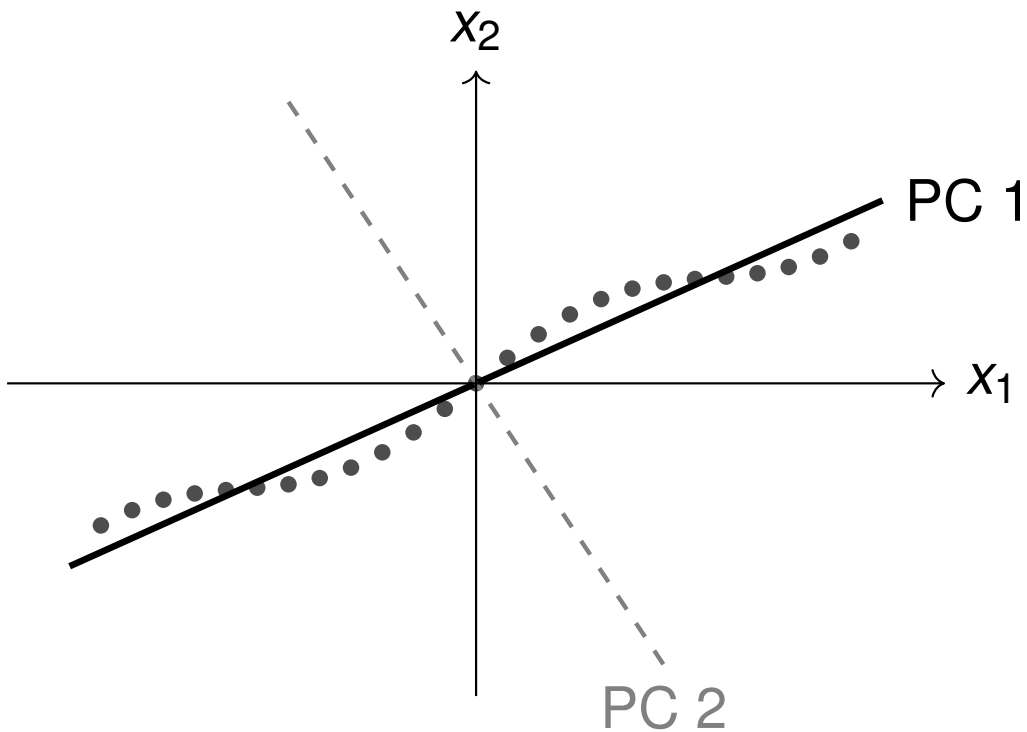
2. Manifold learning

- Los datos viven cerca de una variedad no lineal.
- Se busca “desenrollar” o parametrizar esa estructura.
- Ejemplo: LLE, Isomap, t-SNE, UMAP.



PCA: idea central

PCA busca direcciones ortogonales que capturan la mayor varianza posible.



La primera componente principal es la dirección de máxima varianza; la segunda es ortogonal y captura la mayor varianza restante.

PCA (teoría)[1]

Tenemos datos

$$x_1, \ldots, x_N \in \mathbb{R}^D.$$

Queremos representarlos en un espacio de menor dimensión:

$$M < D.$$

La idea de PCA es encontrar direcciones sobre las cuales los datos tengan la mayor variabilidad posible.

Intuición

La primera componente principal es la dirección donde las proyecciones de los datos quedan más "extendidas".

[1] Pattern Recognition and Machine Learning - Christopher M. Bishop

Datos centrados

Primero calculamos la media muestral:

$$\bar{x} = \frac{1}{N} \sum_{n=1}^{N} x_n.$$

Después trabajamos con los datos centrados:

$$x_n - \bar{x}.$$

La matriz de covarianza muestral es

$$\boxed{S = \frac{1}{N} \sum_{n=1}^{N} (x_n - \bar{x})(x_n - \bar{x})^\top}$$

Varianza proyectada

Queremos que las proyecciones tengan máxima varianza.

La varianza de los datos proyectados es

$$\frac{1}{N}\sum_{n=1}^{N}\left[u_1^\top(x_n-\bar{x})\right]^2.$$

Esta expresión se puede escribir como

$$\boxed{u_1^\top S u_1}$$

Por lo tanto, el problema de PCA en una dimensión es

$$\boxed{\max_{u_1} \quad u_1^\top S u_1 \quad \text{sujeto a} \quad u_1^\top u_1 = 1.}$$

Aparece el problema de eigenvalores

A partir del lagrangiano obtenemos

$$Su_1 = \lambda_1 u_1.$$

Esto significa que u_1 debe ser un **eigenvector** de la matriz de covarianza S.

Además, como $u_1^\top u_1 = 1$, entonces

$$u_1^\top S u_1 = \lambda_1.$$

Conclusión

La varianza proyectada es igual al eigenvalor correspondiente.

Primera componente principal

Como queremos maximizar la varianza proyectada,

$$u_1^\top S u_1 = \lambda_1,$$

debemos elegir el eigenvector asociado al eigenvalor más grande.

$$\boxed{u_1 = \text{eigenvector de } S \text{ asociado al mayor eigenvalor}}$$

Primera componente principal

La primera componente principal es la dirección en la que los datos tienen máxima varianza.

Segunda componente principal

Para encontrar la segunda componente principal buscamos otra dirección que maximice la varianza:

$$\max_{u_2} \quad u_2^\top S u_2.$$

Pero ahora imponemos dos restricciones:

$$u_2^\top u_2 = 1,$$

$$u_2^\top u_1 = 0.$$

Por lo tanto,

$$\max_{u_2} \quad u_2^\top S u_2 \quad \text{sujeto a} \quad \begin{cases} u_2^\top u_2 = 1, \\ u_2^\top u_1 = 0. \end{cases}$$

Lagrangiano para la segunda componente

Tenemos dos restricciones, por lo que introducimos dos multiplicadores de Lagrange:

$$\mathcal{L} = u_2^\top S u_2 + \lambda(1 - u_2^\top u_2) - \alpha\, u_2^\top u_1.$$

Derivando respecto a u_2:

$$2Su_2 - 2\lambda u_2 - \alpha u_1 = 0.$$

Por lo tanto,

$$\boxed{Su_2 = \lambda u_2 + \frac{\alpha}{2} u_1.}$$

Todavía no podemos concluir que u_2 sea un eigenvector: primero debemos mostrar que $\alpha = 0$.

¿Qué ocurre con la restricción de ortogonalidad?

Partimos de

$$Su_2 = \lambda u_2 + \frac{\alpha}{2} u_1.$$

Multiplicamos por $u_1^\top$:

$$u_1^\top S u_2 = \lambda u_1^\top u_2 + \frac{\alpha}{2} u_1^\top u_1.$$

Como

$$u_1^\top u_2 = 0, \qquad u_1^\top u_1 = 1,$$

obtenemos

$$\boxed{u_1^\top S u_2 = \frac{\alpha}{2}.}$$

La simetría de S

La matriz de covarianza S es simétrica $S^\top = S$. Por lo tanto,

$$u_1^\top S u_2 = (S u_1)^\top u_2.$$

Pero ya sabemos que u_1 es un eigenvector ($S u_1 = \lambda_1 u_1$). Entonces,

$$u_1^\top S u_2 = \lambda_1 u_1^\top u_2 = 0.$$

Como también habíamos obtenido

$$u_1^\top S u_2 = \frac{\alpha}{2},$$

necesariamente

$$\boxed{\alpha = 0.}$$

Segunda componente: eigenproblema

Como $\alpha = 0$, la condición del Lagrangiano se reduce a

$$\boxed{Su_2 = \lambda u_2.}$$

Por lo tanto, u_2 también debe ser un **eigenvector** de S.

Además,

$$u_2^\top S u_2 = \lambda.$$

Así que nuevamente la varianza proyectada es el eigenvalor.

Como u_2 debe ser ortogonal a u_1, elegimos el eigenvector correspondiente al segundo eigenvalor más grande:

$$\boxed{Su_2 = \lambda_2 u_2.}$$

¿Y para la tercera, cuarta, . . . componente?

Supongamos que ya encontramos $u_1, u_2, \ldots, u_{k-1}$, con $Su_j = \lambda_j u_j$.

Buscamos u_k que maximice

$$u_k^\top S u_k,$$

sujeto a

$$u_k^\top u_k = 1$$

y a ser ortogonal a todas las componentes anteriores:

$$\boxed{u_k^\top u_j = 0, \qquad j = 1, \ldots, k-1.}$$

Lagrangiano para la componente k

El Lagrangiano es

$$\mathcal{L} = u_k^\top S u_k + \lambda(1 - u_k^\top u_k) - \sum_{j=1}^{k-1} \alpha_j u_k^\top u_j.$$

Derivando respecto a u_k:

$$2Su_k - 2\lambda u_k - \sum_{j=1}^{k-1} \alpha_j u_j = 0.$$

Por lo tanto,

$$Su_k = \lambda u_k + \frac{1}{2} \sum_{j=1}^{k-1} \alpha_j u_j.$$

Debemos mostrar que todos los multiplicadores α_j son cero.

Argumento por inducción

Multiplicamos la ecuación anterior por $u_i^\top$, para cualquier $i < k$:

$$u_i^\top S u_k = \lambda u_i^\top u_k + \frac{1}{2} \sum_{j=1}^{k-1} \alpha_j u_i^\top u_j.$$

Por ortonormalidad,

$$u_i^\top u_k = 0, \qquad u_i^\top u_j = \delta_{ij},$$

por lo que

$$\boxed{u_i^\top S u_k = \frac{\alpha_i}{2}.}$$

Argumento por inducción

Por simetría de S,

$$u_i^\top S u_k = (S u_i)^\top u_k.$$

Por hipótesis, las componentes anteriores son eigenvectores:

$$S u_i = \lambda_i u_i.$$

Entonces,

$$u_i^\top S u_k = \lambda_i u_i^\top u_k = 0.$$

Pero habíamos obtenido

$$u_i^\top S u_k = \frac{\alpha_i}{2}.$$

Por lo tanto, $\boxed{\alpha_i = 0 \qquad \forall\, i < k.}$

Componentes principales

Como todos los multiplicadores α_i son cero,

$$\boxed{Su_k = \lambda_k u_k.}$$

Por lo tanto, cada componente principal es un eigenvector de la matriz de covarianza S.

Ordenando los eigenvalores,

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_D,$$

obtenemos las componentes principales

$$u_1, u_2, \ldots, u_D.$$

Interpretación: u_1 captura la máxima varianza posible. Cada nueva u_k captura la máxima varianza posible sujeta a ser ortogonal a todas las componentes anteriores.

Representación en baja dimensión

Agrupamos las primeras M componentes principales en la matriz

$$U_M = \begin{bmatrix} u_1 & u_2 & \cdots & u_M \end{bmatrix}.$$

La representación reducida de x_n es

$$\boxed{z_n = U_M^\top (x_n - \bar{x})}$$

donde

$$z_n \in \mathbb{R}^M.$$

Así pasamos de D dimensiones a M dimensiones.

Resumen

1. Centramos los datos.
2. Calculamos la matriz de covarianza:

$$S = \frac{1}{N} \sum_{n=1}^{N} (x_n - \bar{x})(x_n - \bar{x})^\top.$$

3. Buscamos direcciones de máxima varianza:

$$\max_u u^\top S u \quad \text{s.a.} \quad u^\top u = 1.$$

4. Esto lleva al problema:

$$Su = \lambda u.$$

5. Tomamos los eigenvectores con eigenvalores más grandes.

PCA con descomposición en valores singulares (SVD)

Hasta ahora calculamos PCA a partir de la matriz de covarianza de los datos centrados X_c.

$$X_c \longrightarrow S = \frac{1}{N} X_c^\top X_c \longrightarrow \text{eigenvalores y eigenvectores de } S.$$

Sin embargo, podemos calcular las componentes principales **directamente a partir de la matriz de datos centrados** utilizando la descomposición en valores singulares (SVD):

$$\boxed{X_{\text{centrada}} = U \Sigma V^\top.}$$

Toda matriz puede descomponerse de esta forma, donde:

- U y V tienen columnas ortonormales.
- Σ contiene los **valores singulares** $\sigma_1, \sigma_2, \ldots$.

Interpretación intuitiva de SVD

Para cualquier matriz $A \in \mathbb{R}^{m \times n}$, la SVD es

$$\boxed{A = U\Sigma V^\top.}$$

Las columnas de V y U forman dos conjuntos de direcciones ortonormales:

$$V = \begin{bmatrix} v_1 & v_2 & \cdots \end{bmatrix}, \qquad U = \begin{bmatrix} u_1 & u_2 & \cdots \end{bmatrix}.$$

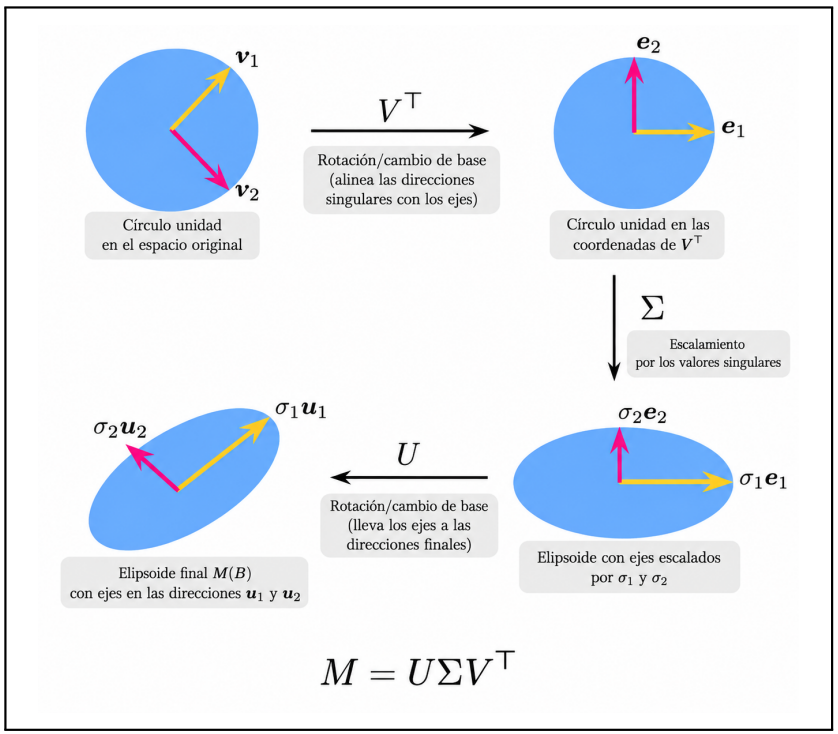
La idea fundamental de SVD es que estas direcciones satisfacen

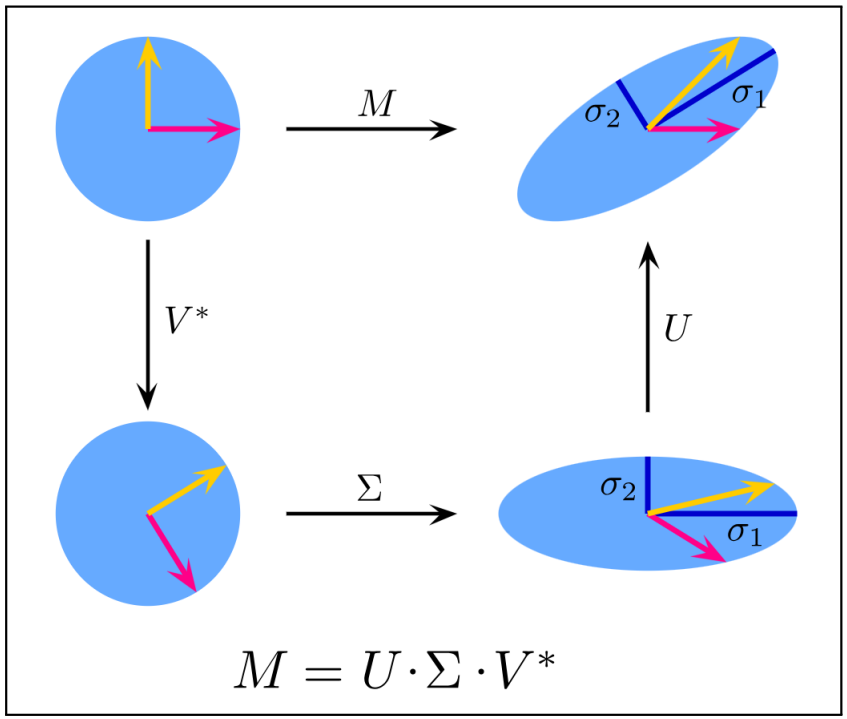
$$\boxed{Av_i = \sigma_i u_i.}$$

Es decir, al aplicar A a la dirección v_i:

$$\boxed{v_i \xrightarrow{A} \sigma_i u_i.}$$

Interpretación geométrica de SVD





Interpretación intuitiva de SVD

La transformación

$$A = U\Sigma V^\top$$

puede interpretarse en tres pasos:

cambio de base $\longrightarrow$ escalamiento $\longrightarrow$ cambio de base

1. $V^\top$ expresa el vector en las direcciones especiales $v_1, v_2, \ldots$
2. Σ escala cada dirección por su valor singular: $\sigma_1, \sigma_2, \ldots$
3. U expresa el resultado en las direcciones $u_1, u_2, \ldots$

Por eso,

$$Av_i = \sigma_i u_i.$$

¿Qué representan los valores singulares?

Los valores singulares se ordenan como

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq 0.$$

Como

$$Av_i = \sigma_i u_i$$

y $\|u_i\| = 1$, entonces

$$\boxed{\|Av_i\| = \sigma_i.}$$

Por lo tanto, σ_i mide cuánto estira A la dirección v_i.

En particular,

$$\boxed{\sigma_1 = \max_{\|x\|=1} \|Ax\|.}$$

¿Por qué SVD nos da las componentes principales?

Partimos de la SVD de los datos centrados:

$$X = U\Sigma V^\top.$$

La matriz de covarianza es

$$S = \frac{1}{N} X^\top X.$$

Sustituyendo la SVD:

$$X^\top X = (U\Sigma V^\top)^\top (U\Sigma V^\top).$$

Entonces,

$$X^\top X = V\Sigma^\top U^\top U\Sigma V^\top.$$

Como las columnas de U son ortonormales, $(U^\top U = I)$,

$$\boxed{S = V\left(\frac{\Sigma^\top \Sigma}{N}\right) V^\top.}$$

SVD y PCA

Obtuvimos

$$S = V\left(\frac{\Sigma^\top \Sigma}{N}\right)V^\top.$$

Esto tiene exactamente la forma de una eigendescomposición:

$$S = V\Lambda V^\top.$$

Por lo tanto, las columnas de V son los eigenvectores de S.
Es decir, $v_1, v_2, \ldots =$ direcciones de las componentes principales.

Además, $\lambda_i = \dfrac{\sigma_i^2}{N}$.

Interpretación: V nos da las **direcciones principales** y los valores singulares de Σ determinan cuánta **varianza** corresponde a cada dirección.

¿Por qué utilizar SVD para PCA?

Una posibilidad es calcular explícitamente la matriz de covarianza $S = \frac{1}{N} X^\top X$, y después obtener sus eigenvalores y eigenvectores.

Sin embargo, podemos aplicar SVD directamente:

$$X = U \Sigma V^\top.$$

Esto tiene varias ventajas:

- No es necesario construir explícitamente la matriz $X^\top X$.
- Suele ser **numéricamente más estable**.
- Las componentes principales se obtienen directamente de las columnas de V.
- Si solo necesitamos algunas componentes, existen métodos eficientes como *truncated SVD* o *randomized SVD*.

Ejemplo

```
import numpy as np
import matplotlib.pyplot as plt

angle = np.pi / 5
stretch = 5
m = 200

rng = np.random.default_rng(seed=3)
X_line = rng.standard_normal((m, 2)) / 10
X_line = X_line @ np.array([[stretch, 0], [0, 1]]) # stretch
X_line = X_line @ [[np.cos(angle), np.sin(angle)],
                   [np.sin(angle), np.cos(angle)]] # rotate

plt.scatter(X_line[:,0],X_line[:,1])

X_centered = X_line - X_line.mean(axis=0)
U, s, Vt = np.linalg.svd(X_centered)

c1 = Vt[0] # primera componente principal
c2 = Vt[1] # segunda componente principal
```

Proyectar a d dimensiones

Si W_d contiene las primeras d componentes principales, entonces

$$X_{d\text{-proj}} = XW_d.$$

- X: matriz original centrada.
- W_d: matriz con las primeras d direcciones principales.
- $X_{d\text{-proj}}$: representación reducida.

Interpretación

Cada nueva característica es una combinación lineal de las características originales.

Ejemplo

```
X_proy = X_centered@Vt[:1].T
plt.scatter(X_proy,np.zeros(m))
```

PCA en Scikit-Learn

```
from sklearn.decomposition import PCA

pca = PCA(n_components=1)
X1D = pca.fit_transform(X_line)
```

Scikit-Learn centra los datos automáticamente.

```
pca.components_ # componentes principales
pca.explained_variance_ratio_ # varianza explicada por componente
```

Varianza explicada

La razón de varianza explicada indica qué proporción de la variabilidad total se conserva en cada componente.

Ejemplo:

$$\texttt{explained_variance_ratio_} = [0{,}82,\ 0{,}11,\ 0{,}07].$$

- PC1 conserva aproximadamente 82 % de la varianza.
- PC2 conserva aproximadamente 11 %.
- PC3 conserva aproximadamente 7 %.

Elegir el número de dimensiones

Una práctica común es conservar, por ejemplo, 95 % de la varianza.

```
from sklearn.decomposition import PCA
import numpy as np

pca = PCA()
pca.fit(X_train)

cumsum = np.cumsum(pca.explained_variance_ratio_)
d = np.argmax(cumsum >= 0.95) + 1
```

O directamente:

```
pca = PCA(n_components=0.95)
X_reduced = pca.fit_transform(X_train)
pca.n_components_
```

Ejemplo Dígitos

```
from sklearn.decomposition import PCA
from sklearn.datasets import fetch_openml
import numpy as np

mnist = fetch_openml('mnist_784', as_frame=False)
X_train, y_train = mnist.data[:60_000], mnist.target[:60_000]
X_test, y_test = mnist.data[60_000:], mnist.target[60_000:]

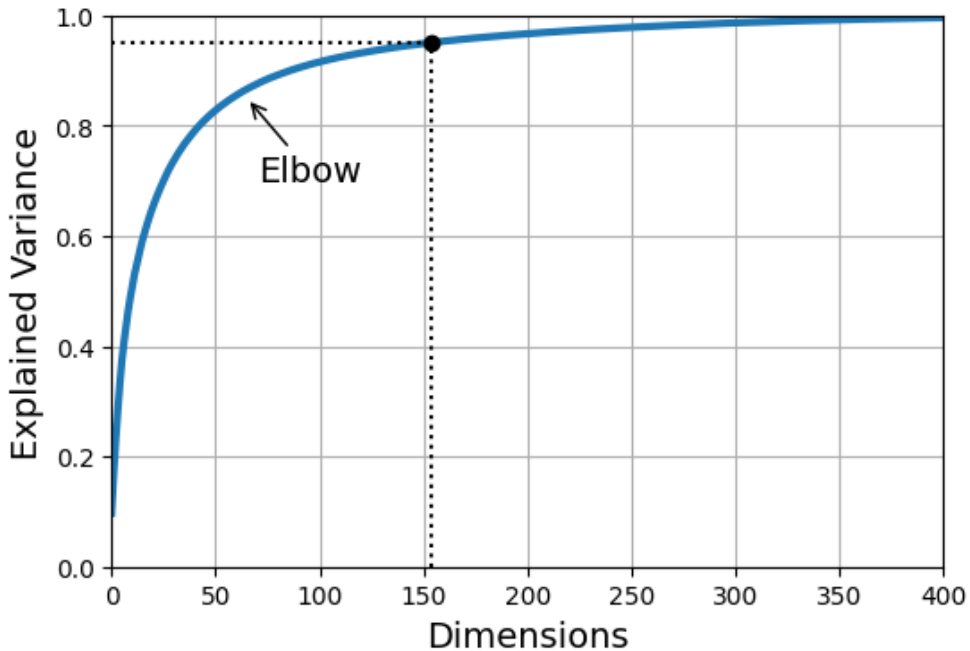
pca = PCA()
pca.fit(X_train)
cumsum = np.cumsum(pca.explained_variance_ratio_)
d = np.argmax(cumsum >= 0.95) + 1 # d equals 154
```

o directamente

```
pca = PCA(n_components=0.95)
X_reduced = pca.fit_transform(X_train)
```

Curva de varianza acumulada

```
import matplotlib.pyplot as plt
plt.plot(cumsum)
plt.grid()
```



Para visualización se eligen 2 o 3 dimensiones; para modelado se puede tratar *d* como hiperparámetro.

PCA dentro de un pipeline

Cuando PCA es preprocesamiento para clasificación, el número de componentes puede ajustarse con validación cruzada.

```
from sklearn.decomposition import PCA
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV
from sklearn.pipeline import make_pipeline

clf = make_pipeline(
    PCA(random_state=42),
    RandomForestClassifier(random_state=42)
)

param_distrib = {
    "pca__n_components": np.arange(10, 80),
    "randomforestclassifier__n_estimators": np.arange(50, 500)
}

rnd_search = RandomizedSearchCV(clf, param_distrib, n_iter=10, cv=3,
                                random_state=42)
rnd_search.fit(X_train[:1000], y_train[:1000])
```

PCA para compresión

Después de aplicar PCA, los datos pueden ocupar mucho menos espacio.

Ejemplo con MNIST:

- Cada imagen original tiene 784 variables.
- Si preservamos el 95 % de la varianza, PCA reduce los datos a 154 variables.
- El tamaño queda en menos del 20 % del original.

La idea es conservar la mayor parte de la información útil, pero con menos dimensiones.

Compresión y reconstrucción

PCA proyecta los datos a un espacio de menor dimensión:

$$X \longrightarrow X_{d\text{-proj}} = XW_d.$$

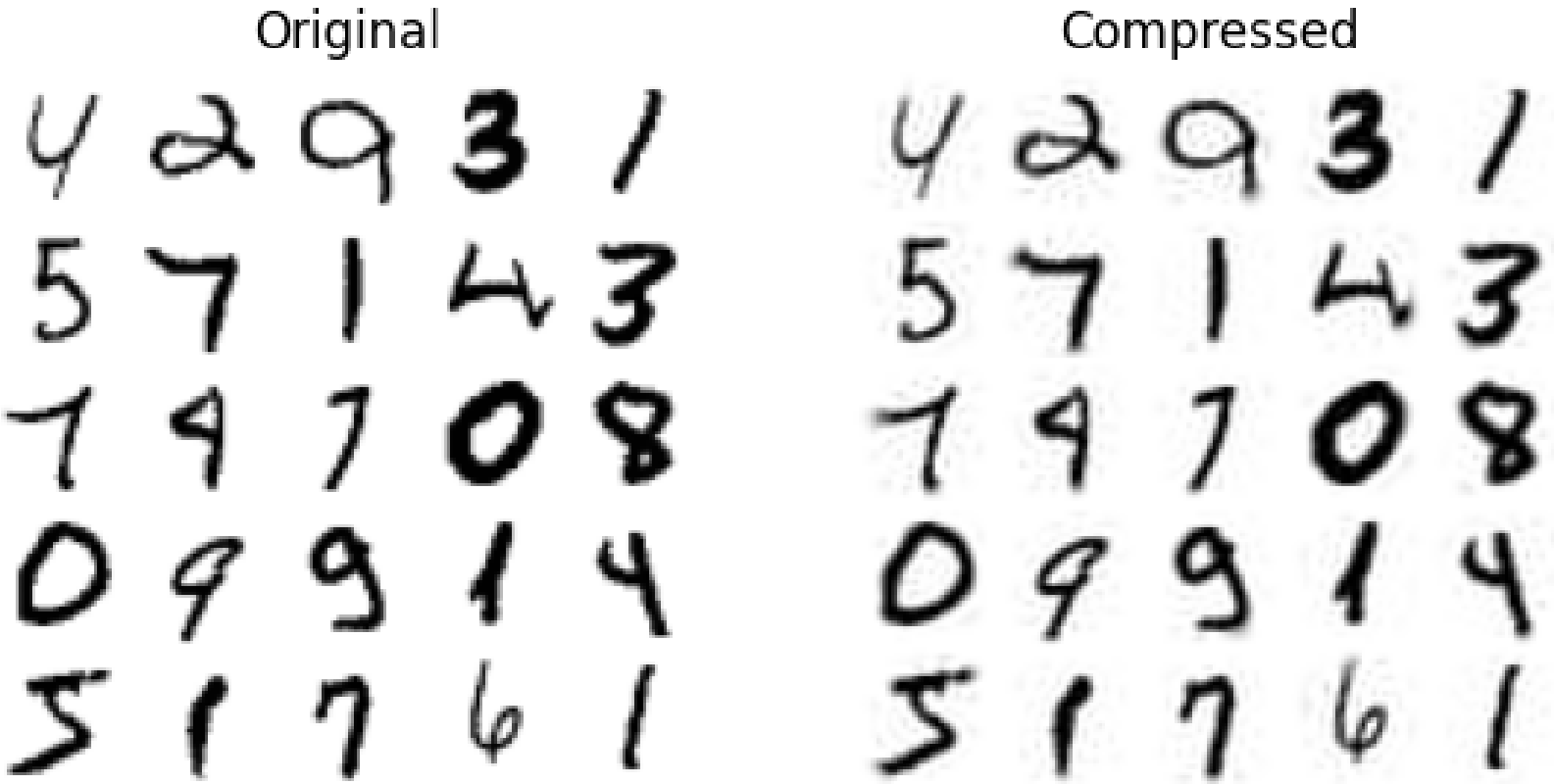
Después podemos reconstruir una aproximación de los datos originales usando la transformación inversa:

$$X_{\text{recovered}} = X_{d\text{-proj}} W_d^\top$$

Pero la reconstrucción no es exacta, porque PCA descartó parte de la información.

```
X_recovered = pca.inverse_transform(X_reduced)
```

Ejemplo Dígitos



Ejemplo Dígitos

Con el ejemplo de la diapositiva 37:

```
plt.figure(figsize=(7, 4))
for idx, X in enumerate((X_train[::2100], X_recovered[::2100])):
    plt.subplot(1, 2, idx + 1)
    plt.title(["Original", "Compressed"][idx])
    for row in range(5):
        for col in range(5):
            plt.imshow(X[row * 5 + col].reshape(28, 28), cmap="binary",
                       vmin=0, vmax=255, extent=(row, row + 1, col, col + 1)
                           )
            plt.axis([0, 5, 0, 5])
            plt.axis("off")
```

Error de reconstrucción

Al comprimir y luego descomprimir los datos, no recuperamos exactamente el dataset original.

El **error de reconstrucción** mide qué tan lejos está la reconstrucción de los datos originales.

$$\text{error} = \frac{1}{m} \sum_{i=1}^{m} \left\| x^{(i)} - x^{(i)}_{\text{recovered}} \right\|^2$$

Mientras más componentes conservamos, menor será el error de reconstrucción.

PCA en Scikit-Learn

```
from sklearn.decomposition import PCA

pca = PCA(n_components=0.95)
X_reduced = pca.fit_transform(X_train)

X_recovered = pca.inverse_transform(X_reduced)
```

- `fit_transform()` reduce la dimensionalidad.
- `inverse_transform()` reconstruye una aproximación.
- La reconstrucción suele ser parecida, pero con cierta pérdida de detalle.

Randomized PCA

Para datasets grandes, calcular PCA con SVD completa puede ser costoso.

Randomized PCA usa un algoritmo estocástico para aproximar los primeros d componentes principales.

Ventaja principal:

$$O(md^2) + O(d^3)$$

en lugar de aproximadamente

$$O(mn^2) + O(n^3)$$

Es especialmente útil cuando:

$$d \ll n$$

Randomized PCA en código

```
from sklearn.decomposition import PCA

rnd_pca = PCA(
    n_components=154,
    svd_solver="randomized",
    random_state=42
)

X_reduced = rnd_pca.fit_transform(X_train)
```

En Scikit-Learn, `svd_solver="auto"` (default) decide automáticamente qué método usar.

Si queremos forzar SVD completa:

```
pca = PCA(n_components=154, svd_solver="full")
```

Incremental PCA

Una limitación de PCA estándar es que necesita que todo el dataset esté en memoria.

Incremental PCA permite procesar los datos por mini-batches.

Esto es útil cuando:

- El dataset es demasiado grande para caber en memoria.
- Queremos aplicar PCA de forma incremental.
- Los datos llegan poco a poco.

Incremental PCA en código

```
from sklearn.decomposition import IncrementalPCA
import numpy as np

n_batches = 100
inc_pca = IncrementalPCA(n_components=154)

for X_batch in np.array_split(X_train, n_batches):
    inc_pca.partial_fit(X_batch)

X_reduced = inc_pca.transform(X_train)
```

La diferencia clave es usar:

`partial_fit()`

en lugar de entrenar con todo el dataset a la vez.

Random Projection

Cuando los datos tienen muchísimas dimensiones, PCA puede seguir siendo lento.

Random Projection proyecta los datos usando una matriz aleatoria:

$$X_{\text{reduced}} = XP^{\top},$$

donde cada entrada es generada con una distribución Gaussiana con media 0 y varianza $1/d$, con d el número de dimensiones a conservar.

Aunque suena sorprendente, una proyección aleatoria puede preservar bastante bien las distancias entre puntos.

Esto se justifica con el lema de Johnson–Lindenstrauss.

Número de dimensiones en Random Projection

La cantidad mínima de dimensiones depende de:

- m: número de instancias.
- ε: tolerancia máxima de distorsión.

Una cota típica es:

$$d \geq \frac{4 \log(m)}{\frac{1}{2}\varepsilon^2 - \frac{1}{3}\varepsilon^3}$$

Observación importante:

d no depende directamente de n.

Random Projection en Scikit-Learn

```
from sklearn.random_projection import (
    johnson_lindenstrauss_min_dim,
    GaussianRandomProjection
)

m, eps = 5000, 0.1
d = johnson_lindenstrauss_min_dim(m, eps=eps)

gaussian_rnd_proj = GaussianRandomProjection(
    eps=eps,
    random_state=42
)

X_reduced = gaussian_rnd_proj.fit_transform(X)
```

Gaussian vs Sparse Random Projection

Scikit-Learn incluye dos variantes:

Método	Matriz aleatoria	Ventaja
GaussianRandomProjection	Densa	Simple y directa
SparseRandomProjection	Dispersa	Menos memoria, más rápida

Ambos métodos preservan aproximadamente las distancias, pero utilizan una matriz de proyección distinta.
Para datasets grandes o dispersos, suele convenir usar `SparseRandomProjection`.

LLE: Locally Linear Embedding

LLE es una técnica no lineal de reducción de dimensionalidad.

A diferencia de PCA o Random Projection:

LLE no se basa en una proyección lineal global.

La idea:

1. Para cada punto, encontrar sus vecinos más cercanos.
2. Expresar cada punto como combinación lineal de sus vecinos.
3. Buscar una representación de baja dimensión que preserve esas relaciones locales.

Intuición de LLE

LLE es especialmente útil cuando los datos están sobre una variedad no lineal.

Ejemplo clásico:

Swiss roll

LLE intenta “desenrollar” la variedad preservando relaciones locales.

Pero no siempre preserva bien las distancias globales.

LLE en código

```
from sklearn.datasets import make_swiss_roll
from sklearn.manifold import LocallyLinearEmbedding
import matplotlib.pyplot as plt

X_swiss, t = make_swiss_roll(n_samples=1000, noise=0.2, random_state=42)

lle = LocallyLinearEmbedding(n_components=2, n_neighbors=10, random_state=42)
X_unrolled = lle.fit_transform(X_swiss)

# Crear figura
fig = plt.figure(figsize=(12, 5))
# Swiss Roll original (3D)
ax1 = fig.add_subplot(1, 2, 1, projection="3d")
ax1.scatter(X_swiss[:, 0],X_swiss[:, 1],X_swiss[:, 2],c=t,cmap="hot")
ax1.set_xlabel("$x_1$")
ax1.set_ylabel("$x_2$")
ax1.set_zlabel("$x_3$")
ax1.set_title("Swiss Roll original")
# Swiss Roll despus de LLE (2D)
ax2 = fig.add_subplot(1, 2, 2)
ax2.scatter(X_unrolled[:, 0],X_unrolled[:, 1],c=t,cmap="hot")
ax2.set_xlabel("$z_1$")
ax2.set_ylabel("$z_2$", rotation=0)
ax2.grid(True)
ax2.set_title("Swiss Roll usando LLE")

plt.tight_layout()
plt.show()
```

LLE: paso 1

Para cada instancia $x^{(i)}$, buscamos pesos $w_{i,j}$ que reconstruyan el punto usando sus k vecinos más cercanos:

$$\hat{W} = \arg\min_{W} \sum_{i=1}^{m} \left\| x^{(i)} - \sum_{j=1}^{m} w_{i,j} x^{(j)} \right\|^2$$

sujeto a:

$$w_{i,j} = 0 \quad \text{si } x^{(j)} \text{ no es } k \text{ vecino más cercano de } x^{(i)}$$

$$\sum_{j=1}^{m} w_{i,j} = 1$$

LLE: paso 2

Después se buscan puntos $z^{(i)}$ en baja dimensión que preserven esos pesos:

$$\hat{Z} = \arg\min_{Z} \sum_{i=1}^{m} \left\| z^{(i)} - \sum_{j=1}^{m} \hat{w}_{i,j} z^{(j)} \right\|^2$$

Es decir:

preservar relaciones locales $\implies$ representación de baja dimensión

Otras técnicas de reducción de dimensionalidad

Técnica	Idea principal
MDS (Multidimensional Scaling)	Reduce la dimensionalidad intentando preservar las distancias entre las instancias, buscando una representación de menor dimensión con distancias similares.
Isomap (Grafo)	Conecta cada instancia con sus vecinos más cercanos para formar un grafo y preserva las distancias geodésicas, es decir, las distancias a lo largo del grafo.
t-SNE (t-distributed Stochastic Neighbor Embedding)	Mantiene juntas las instancias similares y separa las instancias diferentes; se utiliza principalmente para **visualización**, especialmente de clusters.
LDA (Linear Discriminant Analysis)	Método supervisado que busca los ejes que mejor separan las clases; estos ejes pueden utilizarse para reducir la dimensionalidad antes de clasificar.
UMAP (Uniform Manifold Approximation and Projection)	Técnica de reducción no lineal similar a t-SNE que intenta preservar tanto la estructura local como la estructura global de los datos. Para **visualización**.

¿Cuándo usar cada técnica?

- **PCA**: reducción lineal, compresión y preprocesamiento.
- **Randomized PCA**: PCA más rápida cuando $d \ll n$.
- **Incremental PCA**: datasets grandes o por mini-batches.
- **Random Projection**: datos de altísima dimensión.
- **LLE**: variedades no lineales pequeñas o medianas.
- **t-SNE** / **UMAP**: visualización en 2D o 3D.

Resumen

- PCA puede usarse como técnica de compresión.
- La reconstrucción aproximada introduce error.
- Randomized PCA acelera el cálculo de componentes principales.
- Incremental PCA permite trabajar con datos que no caben en memoria.
- Random Projection es simple, rápida y útil en alta dimensión.
- LLE captura relaciones locales en datos no lineales.
- MDS, Isomap, t-SNE, LDA y UMAP son alternativas para distintos objetivos.

¡Gracias!